

Databricks 新一代 RAG

从企业文档接入到生产级 AI 应用的完整实施指南

版本：1.0编写日期：2026 年 9 月 28 日参考：用户提供的 LinkedIn 帖子、Databricks 官方文档

1. 执行摘要

这篇方案讨论一种端到端的企业 RAG（Retrieval-Augmented Generation，检索增强生成）架构：先用 Lakeflow Connect 或文件接口把 SharePoint、Google Drive、S3、工作区文件等内容同步到 Unity Catalog；再用 ai_parse_document 将 PDF、图片、表格和扫描内容解析成结构化结果；随后使用 ai_prep_search 生成适合检索的语义分块和元数据；最后建立 Databricks AI Search / Vector Search 索引，并由 Databricks App 或模型服务端点完成检索、提示词拼接和回答。

关键思想不是“把所有文件直接塞给大模型”，而是把文档处理、权限、检索、生成、评估和监控做成可复用的数据产品。

2. 这条方案解决什么问题

3. 总体架构

[SharePoint / Google Drive / S3 / Upload]



3.1 分层设计

层职责典型对象

Source保存原始文件和同步元数据，尽量不可变Volume、FILE 表、原始 Delta 表

Parse抽取文本、页面、表格、图像描述和文档元素ai_parse_document 结果

Prepare规范化、去重、分块、加元数据、准备向量ai_prep_search、chunk Delta 表

Retrieve向量/关键词/混合检索、过滤、重排AI Search / Vector Search index

Serve身份验证、提示词、回答、引用和反馈Databricks App、Serving endpoint

Evaluate离线质量、在线延迟、安全与成本监控评估集、MLflow tracing、系统表

4. 前置准备

创建或选择 Unity Catalog catalog/schema，并确定数据所有者和生产服务主体。确认 Databricks Runtime、SQL Warehouse 和 AI Functions 可用性；不同 AI Function 的运行时要求可能不同，部署前以当前官方文档为准。建立源系统到用户/组的身份映射；不要等到检索层才临时补权限。准备至少 20–50 个真实问题的黄金评估集，每个问题包含期望答案、来源文档和可接受的引用范围。决定数据新鲜度：小时级、日级还是事件触发；决定删除、重命名和权限变更如何传播。

5. 第一步：接入原始文档

5.1 选择接入方式

方式适合场景注意事项

Lakeflow Connect已有 SaaS/文件源，需要持续增量同步确认连接器、权限、删除语义和刷新频率

Unity Catalog Volume受控上传、批处理、PoC需建立上传审计、命名和病毒扫描流程

FILE 类型列以表的形式统一保存文件与文件元数据检查当前区域/运行时支持和预览限制

对象存储 + Auto Loader大规模文件流入、事件驱动自己维护 schema、幂等、删除和坏文件处理

5.2 原始表建议

```
CREATE TABLE IF NOT EXISTS rag.raw_documents (
```

```
  document_id STRING,
```

```
  source_uri STRING,
```

```
  source_system STRING,
```

```
  source_path STRING,
```

```
  file_name STRING,
```

```
  mime_type STRING,
```

```
  file_size BIGINT,
```

```
  source_modified_at TIMESTAMP,
```

```
  content_hash STRING,
```

```
  acl_group_ids ARRAY<STRING>,
```

```
  ingest_status STRING,
```

```
  ingested_at TIMESTAMP,
```

```
  file FILE
```

) USING DELTA;

生产环境应以 content_hash 和源系统的稳定 ID 做幂等键；将删除和权限变化作为事件或状态，而不是仅依赖文件更新时间。

6. 第二步：解析文档

ai_parse_document 的目标是把非结构化文件变成可处理的结构化结果。它尤其适合 PDF、扫描件、图片、表格和包含复杂版面的材料。解析输出应保留原始文档 ID、页码、元素类型、文本、坐标或布局信息（若返回），以便回答时生成可追溯引用。

```
CREATE OR REPLACE TABLE rag.parsed_documents AS
```

```
SELECT
```

```
document_id,
```

```
source_uri,
```

```
source_modified_at,
```

```
ai_parse_document(file) AS parsed_doc
```

```
FROM rag.raw_documents
```

```
WHERE ingest_status = 'READY';
```

6.1 解析后的规范化结构

```
document_id | page_no | element_no | element_type
```

```
    | text | table_json | image_ref | parser_version
```

```
    | source_uri | source_modified_at | acl_group_ids
```

正文：按标题、段落和列表保存。表格：保留行列关系，不要只转成无分隔纯文本。图片：若业务重要，额外生成图像描述或 OCR 文本，并明确这是推断结果。页码：必须进入 chunk 元数据，便于 UI 展示“第 N 页”。版本：保存解析器版本和源文件 hash，避免内容变化后无法解释结果差异。

7. 第三步：准备语义检索数据

ai_prep_search 用于把解析内容转换为更适合搜索的语义表示，通常包括清洗、分块、上下文补充和搜索元数据。分块不是越小越好：过小会丢失定义和条件，过大则会降低召回的精确度并增加生成成本。

7.1 分块策略

7.2 示例 SQL

```
CREATE OR REPLACE TABLE rag.search_chunks AS
```

```
SELECT
```

```
document_id,
```

```
chunk_id,
```

```
chunk_text,
```

```
named_struct(
```

```
    'source_uri', source_uri,
```

```

'file_name', file_name,
'page_no', page_no,
'source_modified_at', source_modified_at,
'acl_group_ids', acl_group_ids,
'document_version', document_version
) AS metadata
FROM ai_prep_search(
  TABLE rag.normalized_elements,
  map('chunking_strategy', 'semantic',
    'include_context', 'true')
);

```

说明：函数签名、参数名和可用选项会随 Databricks 版本和区域更新；上线前请按工作区文档验证。上述代码是流程模板，不应直接视为固定 API 合约。

8. 第四步：建立 AI Search / Vector Search

索引应以 chunk 表为输入，并同步正文、向量和可过滤元数据。推荐至少支持：按文档 ID、源系统、时间、部门、权限组、语言和文档状态过滤。对企业问答而言，纯向量检索通常不足以处理产品编号、合同号、错误码等精确字符串，因此应评估关键词与语义检索的混合策略。

8.1 索引字段建议

8.2 检索流程

- 从用户身份得到组/角色集合。
- 将问题规范化，但不要删除专有名词、编号和日期。
- 执行向量或混合检索，并带上 ACL、租户、状态过滤。
- 取较大的候选集，再用重排或业务规则选出最终上下文。
- 去除重复 chunk，按文档版本和来源排序。
- 把引用元数据与文本一起传给生成模型。

9. 第五步：构建问答应用

9.1 最小提示词模板

系统：你是企业知识助手。只依据“检索上下文”回答。

如果上下文不足，明确说“资料不足”，不要猜测。

回答结论后列出引用：[文件名，第 X 页，版本 Y]。

涉及权限、金额、合规或安全时，提醒用户以正式流程为准。

用户问题：{question}

检索上下文：

{retrieved_chunks}

9.2 UI 必备能力

显示答案依据：文件名、页码、版本、更新时间和可点击源链接。显示“不确定/资料不足”状态，而不是用肯定语气填空。支持过滤器：部门、文档类型、时间范围和业务域。保留用户反馈、检索结果和最终答案，用于评估；敏感内容须脱敏。在流式输出时先显示检索状态，再显示答案，避免用户误以为模型已完成验证。

10. 典型业务场景

场景输入检索/生成要求主要风险

员工政策助手制度、HR FAQ按地区/员工类型过滤，显示生效日期过期政策、个性化权限

客服知识助手产品手册、故障库错误码精确匹配 + 语义召回错误操作、版本错配

销售赋能产品资料、竞品分析按产品版本和市场过滤机密外泄、虚构卖点

合同检索合同、条款模板条款级引用、版本与签署状态法律结论、权限泄露

运维助手Runbook、告警文档按环境和服务过滤，输出步骤和前置条件高风险自动执行

研究/报告问答报告、论文、表格表格结构、时间范围和来源完整性统计误读、引用不完整

11. 安全、治理与合规

最小权限：源连接、解析任务、索引服务和应用服务主体分别授予必要权限。检索前授权：ACL 过滤必须在检索阶段生效；仅在提示词中要求模型“不回答机密内容”是不够的。租户隔离：多租户场景优先使用可验证的租户字段和服务端过滤；不要依赖客户端传入的租户 ID。数据生命周期：定义删除、保留、撤回、权限变更和索引同步 SLA。秘密管理：API key、连接凭据和 OAuth secret 放入受控 secret 管理，不进 notebook、日志或 prompt。审计：记录谁在何时对哪个数据范围提出问题、检索了哪些 chunk、生成了什么答案；按公司政策控制保留期限。提示词注入：把文档内容视为不可信输入；模型必须遵循系统规则，不执行文档中的工具调用或越权指令。

12. 评估与生产验收

12.1 离线指标

指标含义建议验收问题

Recall@k正确来源是否进入候选集前 5 个结果是否包含黄金来源

Context precision上下文是否少噪声检索内容是否与问题直接相关

Faithfulness答案是否被上下文支持每个关键结论是否有依据

Answer correctness最终回答是否正确、完整专家评分或标准答案比对

引用正确率引用是否真的支持结论链接、页码、版本能否复核

12.2 必测用例

同一主题存在多个版本时，是否优先生效版本。用户无权访问文档时，是否既不返回内容也不泄露文件名。问题包含错别字、别名、错误码、编号和日期时的召回情况。跨页表格、扫描 PDF、图片、双语文档和空白页。文档删除或权限撤回后，索引和回答多久停止使用旧内容。模型服务不可用、索引延迟、解析失败时，是否有可理解的降级提示。

13. 性能与成本优化

只对新增或内容 hash 变化的文件重新解析和分块。为高频问题缓存安全的检索结果，但要把权限和文档版本纳入缓存键。候选集、chunk 长度和重排数量通过评估集调优，不要盲目增加上下文。大表格和长文档采用分层摘要：先检索摘要，再按需读取原文片段。区分批处理、准实时和在线请求预算，避免在同步链路里同步执行昂贵生成。用查询

日志、模型 token、索引刷新和任务运行时建立成本看板。

14. 交付路线图

阶段交付物退出标准

0. 设计数据地图、权限模型、评估集、架构图明确数据所有者、SLA 和风险边界

1. PoC单一来源、解析、chunk、索引、问答 Demo黄金问题中大部分可追溯回答

2. 试点增量同步、ACL、反馈、监控、故障处理业务用户可用，权限测试通过

3. 生产Bundle/CI-CD、服务主体、评估门禁、审计可重复部署，达到延迟、质量和成本 SLO

4. 扩展多源、多语言、多租户、Agent/工具调用每个扩展场景独立完成安全和评估

15. 上线检查清单

☐ 所有数据集有 owner、分类、保留策略和访问矩阵 ☐ 原始文件、解析结果、chunk 和索引都能按版本追溯 ☐ 增量、删除、重命名和权限撤回已测试 ☐ 检索过滤在服务端执行，并有越权自动化测试 ☐ 回答强制提供引用或明确标记资料不足 ☐ 已建立 Recall、faithfulness、正确率、延迟和成本指标 ☐ 失败任务、索引落后、模型超时和配额错误有告警 ☐ 部署由 DAB/CI-CD 管理，不依赖个人 notebook 状态 ☐ 生产日志脱敏，prompt、chunk 和用户反馈按政策保留

16. 参考资料与说明

本文基于用户提供的 LinkedIn 帖子主题进行归纳，并结合 Databricks 官方文档组织为实施指南。文中的 SQL 主要用于解释架构和数据流；具体函数签名、运行时要求、区域可用性、Preview/Beta 状态以及产品命名可能变化，实施时应在目标工作区版本上重新验证。

Databricks 文档索引：<https://docs.databricks.com/llms.txt>

AI Functions 概览：<https://docs.databricks.com/aws/en/large-language-models/ai-functions>

ai_parse_document：https://docs.databricks.com/aws/en/sql/language-manual/functions/ai_parse_document

ai_prep_search：https://docs.databricks.com/aws/en/sql/language-manual/functions/ai_prep_search

AI Search / Vector Search：<https://docs.databricks.com/aws/en/generative-ai/ai-search/>

Lakeflow Connect：<https://docs.databricks.com/aws/en/ingestion/lakeflow-connect>

Databricks Apps：<https://docs.databricks.com/aws/en/dev-tools/databricks-apps/>

Unity Catalog：<https://docs.databricks.com/aws/en/data-governance/unity-catalog/>

用户提供的 LinkedIn 帖子：https://www.linkedin.com/posts/thomas-hass-53b39b19a_

